Azizbek Arzikulov

AI / Full-Stack Product Engineer

Founder-style ownership for early-stage products, data systems, and integration-heavy builds

AI products | Data pipelines | Platform thinking | Deployment

I am strongest when the product is still taking shape and somebody has to turn scattered ideas, messy integrations, and evolving requirements into a working system that people can actually use.

In this deck:

- Why the role is a stronger strategic fit than a generic backend position
- Case studies chosen for AI, data, recommendation, and platform ownership
- How I think in early-stage environments where architecture and product move together

Email: azizbek.dev.ac@gmail.com
Phone: +82-10-5631-5231
GitHub: github.com/azizbekdevuz
LinkedIn: https://www.linkedin.com/in/azizbek-arzikulov

Confidential candidate material

Operating model

How I usually work when a product is still early and messy.

The pattern across my better projects is not “build feature, move on.” It is: define the shape, make it work, then remove the friction that stops it from scaling.

1	**1. Clarify the real decision**	What exactly has to become easier, faster, safer, or more accurate for the user?
2	**2. Build the minimum system boundary**	Data model, API shape, auth boundary, and what belongs in version 1 versus later.
3	**3. Ship a working vertical slice**	I prefer seeing the whole loop run once rather than building disconnected modules forever.
4	**4. Close integration debt**	This is where I usually add value: sessions, edge cases, contracts, background logic, environment mismatches.
5	**5. Harden only what matters next**	Logs, tests, deployment cleanup, and operational guardrails when the system earns them.
6	**6. Keep decisions explainable**	In an early team, speed matters, but so does being able to explain why the system is shaped that way.

Confidential candidate material

Case study 1

Rumi AI — multilingual LLM product with real integration work behind the interface.

This project matters here because the hardest part was not “making a pretty app.” It was making several moving parts behave like one coherent product.

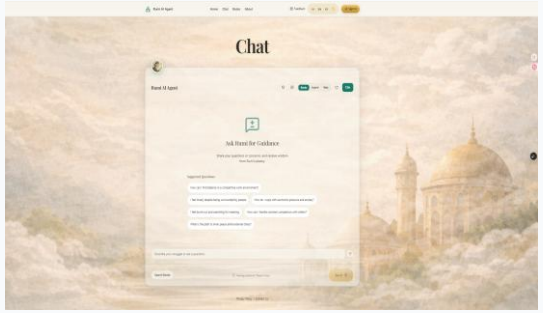
What the product is

A multilingual AI companion inspired by Rumi’s poetry, with retrieval-backed answers, streaming responses, and user-facing guidance grounded in selected source material.

What I actually owned

- Frontend delivery and product behavior across English, Persian (RTL), and Korean surfaces
- BFF-style integration work between frontend routes and Python backend services
- Auth/session cleanup, API contract alignment, and reliability-focused fixes
- Turning separately built pieces into something that felt like one product instead of a demo stitched together late

LLM / RAG | multilingual | integration | research client



Chat page

Shape an AI-backed product so the recommendation logic, user experience, and surrounding system boundaries work together cleanly. Rumi AI is one of my best proofs of that kind of integration-heavy thinking.

Confidential candidate material

Case study 2

Fishlinic — data intake, AI enrichment, and an insight-oriented dashboard.

This is the closest proof that I can work on a product where raw data has to be normalized, interpreted, and made useful to non-technical users.

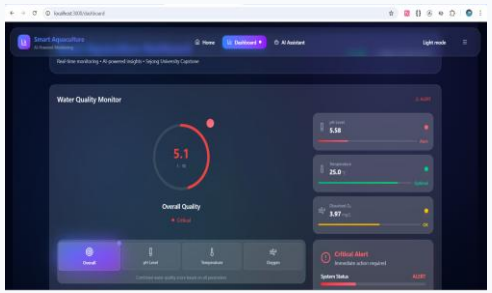
What the system had to do

- Collect real-time water-quality signals from hardware
- Normalize noisy inputs coming from the bridge layer
- Enrich readings with AI-generated quality signals and assistant behavior
- Expose the result through a usable dashboard rather than raw telemetry tables

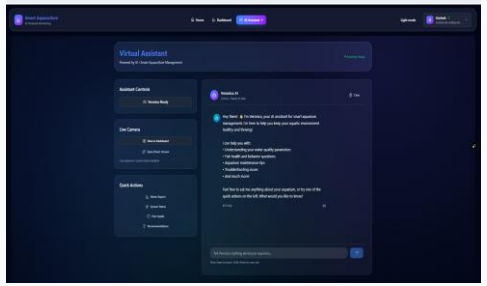
Why I kept it in this portfolio

Engineering pattern is relevant: data comes in imperfectly, the system has to structure it, add machine-generated meaning, and turn it into something people can act on.

real-time data | AI insights | dashboard UX



Dashboard view with live telemetry cards/charts



AI assistant

Technical layers involved

- Next.js web product + Express bridge + Python AI service
- PostgreSQL/Prisma storage for users and telemetry
- Socket-based updates for live visualization
- Cross-team integration with AI and hardware (Arduino + sensors) components

Confidential candidate material

Case study 3

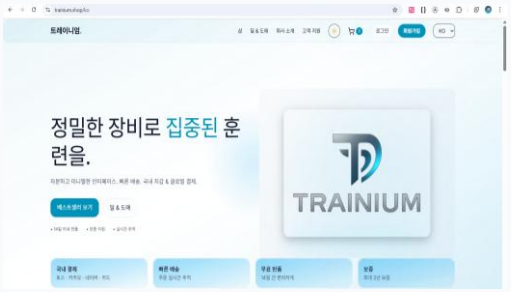
Trainium — long-horizon ownership of a production-minded platform.

This one is less about AI and more about proving I can carry a system myself for a long time: architecture, data model, product complexity, release cleanup, and operations.

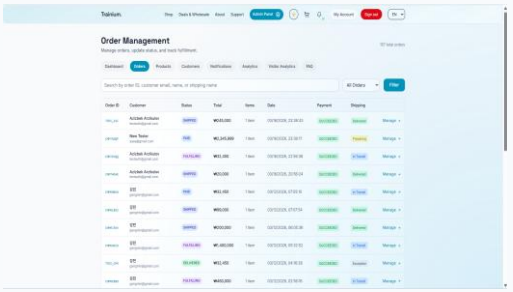
What it demonstrates

- Solo ownership of a real multi-surface product instead of a one-week prototype
- Backend and data design across auth, orders, payments, admin workflows, notifications, and i18n
- Decision-making around what to harden, what to defer, and how to keep the product maintainable as scope grows
- Deployment and runtime responsibility on VPS infrastructure with Docker, Nginx, and SSL/HTTPS

A first CTO/co-founder is not only choosing frameworks. They are inheriting the long-term consequences of those choices. Trainium is my clearest proof that I can live with a product long enough to understand that responsibility.



Homepage



Admin dashboard → order-management screen

One useful detail for this role

Trainium also includes recommendation logic and structured product/customer/admin data. It is not the same kind of matching as I am comfortable with systems where data quality and downstream decisions matter.

Confidential candidate material

Case study 4

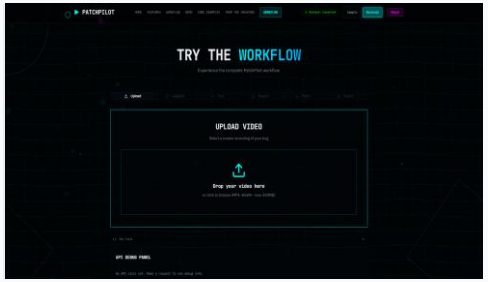
PatchPilot — turning messy input into a reproducible AI workflow.

The reason I kept this project is not hackathon branding. It is the workflow design: take unstructured input, structure it, verify it, and turn it into something actionable.

Pipeline shape

- Video recording goes in
- LLM-based analysis produces structured understanding
- Playwright tests are generated and executed
- The system returns a more grounded patch/report suggestion instead of a blind guess

This project is more than generic LLM usage. It needs structured reasoning around data and useful outputs for parents and operators. PatchPilot shows that I like workflows where the model output becomes part of a larger decision system rather than the whole product by itself.



Workflow page with the multi-step pipeline visible

Generated test/report/patch output screen

What this proves beyond AI hype

- Workflow orchestration across multiple technical steps
- A verification loop instead of pure model optimism
- FastAPI backend thinking around health, errors, and state transitions

Confidential candidate material

Case study 5

ProofBoard — a smaller project, but useful evidence around security and data integrity.

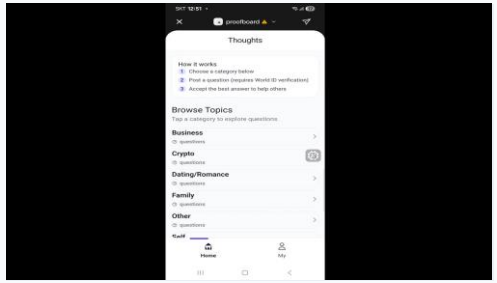
I would not lead my application with this project, but it matters because GrowMatch handles sensitive, trust-dependent data.

What stands out technically

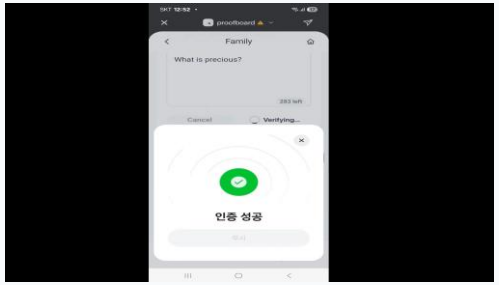
- Server-side verification rather than trusting client-side actions
- Atomic verification + write flow to avoid wasted or inconsistent actions
- Replay protection and action-scoped uniqueness
- Careful transaction thinking around user actions and stored state

Why I kept it in the deck

Sitting near sensitive developmental information, recommendation outcomes, and parent trust. ProofBoard is one of the cleaner examples where I thought carefully about trust boundaries and integrity, not just features.



Q&A board screen



Verification flow UI

Useful signal for a first technical owner

I do not only think about “can the feature work?” I also think about “what breaks trust if the workflow is loose?” That mindset matters more in products touching children and parents than in ordinary consumer apps.

Confidential candidate material

Additional systems

More backend/platform breadth behind the featured projects.

These are not the lead case studies for this application, but to strengthen the picture that I have not only worked on one type of system.

Nestar

NestJS + GraphQL, JWT auth, Mongoose, websockets, scheduler, and test structure. Useful proof that I am comfortable in more strongly structured backend environments as well.

NestJS

Leo Academy

LMS-style platform with sessions/JWT, Stripe (test mode), uploads/media, email flows, and course-management logic. Good signal for operational product breadth.

platform

Burak

Express + TypeScript backend with sessions/JWT, Helmet, rate limiting, Socket.IO, and restaurant-ordering/admin patterns. Smaller project, still real backend signal.

Express

Private repo → code review available upon request

What the overall project mix says

AI product integration, dashboards, recommendation-ish logic, secure action flows, platform ownership, and early-stage shipping.

narrative

Confidential candidate material

Role mapping

Mapping

Mapping	Where I have real proof
AI / LLM product work	Rumi AI, PatchPilot
Data platform thinking	Fishlinic telemetry + AI enrichment, Trainium structured platform data
Recommendation / matching logic	Trainium recommendation patterns; retrieval/ranking behavior in AI products
Early-stage ownership	Trainium, independent builds, hackathon systems, lab integration work
Sensitive-data mindset	ProofBoard trust/integrity patterns; general auth/session boundary work
Technical roadmap / CEO collaboration	Personal ventures + research lab product integration, not formal executive title

Confidential candidate material

Azizbek Arzikulov

AI / Full-Stack Product Engineer

azizbek.dev.ac@gmail.com
+82-10-5631-5231

Seoul, South Korea

Selected links

GitHub — github.com/azizbekdevuz
LinkedIn — linkedin.com/in/azizbek-arzikulov

Rumi AI — github.com/azizbekdevuz/rumi-ai
Fishlinic — github.com/azizbekdevuz/fishlinic
Trainium — trainium.shop
PatchPilot — patchpilot-frontend-beta.vercel.app

Confidential candidate material